MATERIAL DIDÁCTICO · ACCESO A DATOS

UNIDAD DIDÁCTICA 01

Relación 3
Objetos, XML, JSON y repositorios

Ocho ejercicios progresivos sobre serialización, XML, JSON, separación del acceso a datos y un proyecto integrador. No se trabaja el apartado 7 de acceso aleatorio.

- 8 ejercicios
- Apartados 6 · 8 · 9 · 10 · 11
- Sin RandomAccessFile

JAVA · BINARIO · XML · JSON · REPOSITORIOS

Relación 3 · Objetos, XML, JSON y repositorios

Esta relación continúa la Unidad Didáctica 1 después de CRUD y copias de seguridad. Se practican los apartados 6, 8, 9, 10 y 11. El apartado 7 (acceso aleatorio a registros de tamaño fijo) queda expresamente fuera.

> Indicaciones. Cada ejercicio es independiente. Trabaja en su propia carpeta. Cuando un ejercicio incluya archivos de prueba, utiliza los proporcionados en el ZIP de archivos base. Comprueba con **Files.exists(...)** los ficheros que deban existir antes de leerlos. No utilices RandomAccessFile en esta relación.

01 Guardar objetos de inventario en binario

Apartado 6 · Serializable, ObjectOutputStream y listas de objetos

Archivos: Producto.java · GuardarProductosBinario.java · datos/productos.dat

Crea una clase Producto con id, nombre, precio y stock. La clase debe poder serializarse. Después crea una lista con al menos cuatro productos y guarda la lista completa en datos/productos.dat.

- Producto debe implementar Serializable y declarar serialVersionUID.
- Crea la carpeta datos si no existe.
- Guarda la lista completa con ObjectOutputStream y Files.newOutputStream(...).
- No abras ObjectOutputStream en modo APPEND: cada guardado debe sustituir el objeto lista completo.
- Controla IOException y muestra un mensaje comprensible si el guardado falla.

Comprueba: Ejecuta el programa, comprueba con Files.exists(...) que productos.dat se ha creado y verifica que su tamaño es mayor que 0.

02 Recuperar, modificar y volver a guardar

Apartado 6 · ObjectInputStream, comprobación de tipos y persistencia

Archivos: Producto.java · GestionBinariaProductos.java · datos/productos.dat

Parte de un productos.dat ya preparado. Recupera la lista de objetos, muestra todos los productos, pide un ID y un nuevo stock y modifica únicamente el producto encontrado. Después vuelve a guardar la lista completa.

- Antes de leer, comprueba que el fichero existe.
- Lee primero como Object y comprueba que el contenido es una List.
- Comprueba también que cada elemento recuperado es un Producto antes de utilizarlo.
- No reescribas el fichero si el ID no existe o si la lectura no se ha completado correctamente.
- Tras guardar, vuelve a abrir el fichero y comprueba que el nuevo stock persiste.

Comprueba: Cambia el stock de un producto, cierra el programa y ejecútalo de nuevo. Debe recuperarse el valor modificado.

03 Exportar productos a XML con DOM

Apartado 8.1 · Document, Element y Transformer

Archivos: Producto.java · ExportarProductosXml.java · datos/productos.xml

Construye una lista de productos y expórtala a datos/productos.xml mediante DOM. No construyas el XML concatenando cadenas.

- La etiqueta raíz será .
- Cada tendrá el ID como atributo y nombre, precio y stock como elementos hijos.
- Crea el documento con DocumentBuilderFactory y los nodos con createElement(...).
- Escribe el fichero con Transformer, indicando UTF-8 e indentación.
- Prepara la carpeta datos si no existe.

Comprueba: Incluye un nombre con tilde y otro con el carácter '&'. Abre el XML y verifica que ambos se representan correctamente.

04 Importación XML segura

Apartado 8.2 · Parseo, validación y lista temporal

Archivos: Producto.java · ImportarProductosXml.java · datos/productos_correctos.xml · datos/productos_erroneos.xml

Importa productos desde un XML. La importación solo se considerará válida si todos los registros pueden interpretarse correctamente.

- Comprueba que el fichero existe antes de abrirlo.
- Desactiva DTD y entidades externas antes de hacer parse(...).
- Valida ID entero positivo, nombre no vacío, precio mayor o igual que 0 y stock entero mayor o igual que 0.
- Rechaza IDs duplicados dentro del propio XML.
- Carga primero todos los productos en una lista temporal; no sustituyas la lista principal hasta terminar sin errores.

Comprueba: Prueba primero productos_correctos.xml y después productos_erroneos.xml. El segundo debe producir un error sin destruir la lista válida que ya había en memoria.

05 Exportar e importar JSON con Jackson

Apartado 9 · Maven, ObjectMapper y TypeReference

Archivos: pom.xml · src/main/java/Producto.java · src/main/java/ProductosJson.java · datos/productos.json · datos/productos_malformados.json

Crea un proyecto Maven con Java 17 y Jackson. Exporta una lista de productos a JSON y vuelve a recuperarla.

- Añade jackson-databind 2.18.3 al pom.xml.
- Usa ObjectMapper y writerWithDefaultPrettyPrinter() para escribir el JSON.
- Recupera la lista con TypeReference>.
- Valida los datos recuperados antes de sustituir la lista que usa el programa.
- Si el JSON está mal formado, muestra el error y conserva los datos anteriores.

Comprueba: Exporta, modifica manualmente una ciudad... No: en este ejercicio usa productos. Cambia manualmente el stock de un producto en el JSON válido y comprueba que se recupera. Después prueba el fichero malformado.

06 Separar el acceso a datos: repositorio de texto

Apartado 10 · Interfaz e implementación

Archivos: Producto.java · RepositorioProductos.java · RepositorioTexto.java · Main.java · datos/productos.txt

Separa el código del menú del código que accede al fichero. Main no debe utilizar directamente Files.

- Define la interfaz RepositorioProductos con guardarTodos(...) y cargarTodos().
- Implementa RepositorioTexto usando el formato id;nombre;precio;stock en UTF-8.
- Al cargar, valida número de campos y conversiones numéricas.
- Si el fichero no existe, devuelve una lista vacía en lugar de provocar un cierre inesperado.
- Main debe trabajar únicamente contra la interfaz RepositorioProductos.

Comprueba: Busca en Main.java: no debe aparecer Files, BufferedReader, BufferedWriter ni Path.

07 Cambiar de TXT a binario sin cambiar el menú

Apartados 6 y 10 · Implementaciones intercambiables

Archivos: Producto.java · RepositorioProductos.java · RepositorioBinario.java · Main.java · datos/productos.dat

Implementa un segundo repositorio que guarde la misma lista de productos mediante serialización binaria. El programa principal debe conservar las mismas operaciones visibles para el usuario.

- RepositorioBinario debe implementar exactamente la misma interfaz que RepositorioTexto.
- Comprueba Files.exists(...) antes de intentar cargar el fichero.
- Valida el tipo del objeto recuperado y de cada elemento de la lista.
- Main solo debe cambiar la línea en la que se crea el repositorio.
- No dupliques en Main la lógica de lectura o escritura.

Comprueba: Ejecuta primero con RepositorioTexto y después con RepositorioBinario. Las opciones del programa deben seguir siendo las mismas.

08 Proyecto final: gestor de inventario

Apartado 11 · Integración sin acceso aleatorio

Archivos: Proyecto Maven con Main, Producto, repositorios, XML, JSON y copias de seguridad

Desarrolla un gestor de inventario que recuerde los productos entre ejecuciones. Este ejercicio integra lo trabajado en la unidad excepto el apartado 7: no utilices RandomAccessFile ni registros de tamaño fijo.

- Menú mínimo: alta, listado, búsqueda por ID, modificar stock, borrar, guardar/cargar, exportar/importar XML, exportar/importar JSON, copia/restauración y salir.
- No permitas IDs duplicados ni nombres vacíos; precio y stock no pueden ser negativos.
- Las importaciones XML y JSON deben validarse por completo antes de sustituir los datos actuales.
- Antes de restaurar una copia solicita confirmación; comprueba siempre que el fichero de copia existe.
- Separa Main del acceso a datos mediante RepositorioProductos.
- Al cerrar y volver a ejecutar, los productos guardados deben seguir disponibles.

Comprueba: Trabaja con al menos cuatro productos: modifica uno, borra otro, exporta XML y JSON, crea una copia, restaura y reinicia el programa. Prueba también una importación incorrecta y verifica que los datos válidos se conservan.